

Crypto Homework 1

1. Determine the number of keys in an affine cipher over Z_m for $m = 29, 99$, and 1024 .

For $Z_m = 26$, # with Same GCD: 12, Number of Keys: 312

For $Z_m = 29$, # with Same GCD: 28, Number of Keys: 812

For $Z_m = 99$, # with Same GCD: 60, Number of Keys: 5940

For $Z_m = 1024$, # with Same GCD: 512, Number of Keys: 524288

Briefly explain your answers.

To find the number of keys in the above affine cipher given Z_m 's we followed the following three steps:

First, find all numbers, from $1 < x < Z_m$, of the given Z_m that has the greatest common divisor equal to 1 ($\gcd(x, Z_m) = 1$);

Second, I counted all occasion in which $\gcd(x, Z_m) = 1$.

Finally, we multiply the overall number of occasion, in which $\gcd(x, Z_m) = 1$, with its respected Z_m value to obtain the amount of Keys.

Java Code for the Assignment:

Reference file -> GCD.java

```
public class GCD {  
  
    public static void main(String[] args){  
        int a = 1;  
        int i = 0;  
        int b = 26;  
        int keyNum = 0;  
        int x[] = {26, 29, 99, 1024};
```

```
        do{
            a = 1;
            b= x[i];
            keyNum = 0;

            while( a != b){

                int tmp = gcd(a, b);

                if( tmp == 1){
                    //      System.out.println(tmp+"\n");
                    keyNum++;
                }
                //      else
                //      System.out.println("None\n");
                a++;
            }

            System.out.println("For Zm = " + b + ", # with Same GCD: " + keyNum + ", Number of
            Keys: "+ keyNum*b);

        }while( i++ != x.length-1);
    }
    public static int gcd(int a, int b){

        //      System.out.println(" number is : " + a);
        while( a != b){

            if( a > b)
                a -= b;
            else
                b -= a;
        }
        return a;
    }
}
```

2. Perform cryptanalysis of the following two ciphertexts. The first ciphertext was obtained using an affine cipher, and the second ciphertext was obtained using a substitution cipher. Your goal is to find the plaintexts. Both are in English. Also, give the authors and the sources of the plaintexts.

Give a clearly written description of the steps you followed to decrypt both ciphertexts. This should include all statistical analysis and computations you performed. If you wrote code to help you decrypt, you should also include a copy of your source code. (Remember, you can use any programming language you like and there are no style requirements.)

o Affine Cipher

```
XGKTRDCSSVWRKTGHGVWSCRGRDGTGYKCXSFWXGODVWSSGSSGHJGKQRMOCR  
DWQRPKXCRMSRTGXERDOCRDWQRCXSWZGXIGIWQTKEGOCRDWQRFGTWSCRMKXHK  
ZZRDGPCTRQGSWFYKXOCRDWQDCSPCIGSRDCSVTKCSGODCIDOWQZHJGQXYGKX  
CXEFZKRRGTMCFCXSITCJGHWPGTDQYKXKSDGSCSJQRKBQSRRTCJQRGRWRDGYG  
YWTMWFJWKR SOKCXKHWE
```

The steps we followed to decrypt to above the text are as follows:

- 1) Assign numerical values to each letter of the alphabet, which I made by default all capitalized, and store it in a list for future use during the decryption.
- 2) Display and save each character of the text as an object in a list. Each object in this list has a location, counter and name to know where in the text such letter is located, which letter it is and how many times is repeated in the text.
- 3) Create a function named displacement, which first create a list of letters that are not repeated. Then creates a list of objects with those unique letters to be compared with the original list. This way, it checks for how many time each letter is repeated and then increments its counter to keep track of how many time it was seen in the original text. Finally, once all letters where compared the set of unique letters is returned for decryption purpose.
- 4) Create a function named orderInput, which takes the above list of unique objects (letters) organize them according the its counter, time repeated number, to display them in order. Then, it obtains the letters whose counter is greater and display it.
- 5) Finally, create a decrypt method to decrypt the ciphertext with the appropriate alpha and beta. To do this, a list of possible alphas and beta was created and all possibilities where tested. As a result, the plaintext of the above ciphertext was obtained by using the values $\alpha = 25$ and $\beta = 10$ as shown below.

Example Output:

Note: a copy of the original output-file generated by this code is attached.

Descending Order Freqs:

(Letter: R Count: 290)	(Letter: H Count: 70)
(Letter: G Count: 280)	(Letter: J Count: 60)
(Letter: C Count: 230)	(Letter: F Count: 60)
(Letter: S Count: 220)	(Letter: Y Count: 60)
(Letter: W Count: 210)	(Letter: I Count: 50)
(Letter: K Count: 180)	(Letter: Z Count: 50)
(Letter: D Count: 160)	(Letter: M Count: 50)
(Letter: X Count: 150)	(Letter: E Count: 40)
(Letter: Q Count: 130)	(Letter: P Count: 40)
(Letter: T Count: 130)	(Letter: V Count: 40)
(Letter: O Count: 80)	(Letter: B Count: 10)

Max Coincidences is:(Letter: R| Count: 290)

Possible Alpha List : [3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25]

Possible Beta List : [5, 10, 23, 15, 7, 25, 9, 1, 19, 11, 3]

Continue . . .

Output:

NEARTHISSPOTAREDEPOSITEDTHEREMAINSOFOFONEWHOPOSSESSEDBEAUT
YWITHOUTVANITYSTRENGTHWITHOUTINSOLENCECOURAGEWITHOUTFER
OSITYANDALLTHEVIRTUESOFMANWITHOUTHISVICESTHISPRAISEWHICHW
OULD BE UNMEANINGFLATTERYIFINSCRIBEDOVERHUMANASHESISBUTAJU
STTRIBUTETOTHEMEMORYOFBOATSWAINADOG

Inverse of alpha25 is : 25 and Beta is10

Output:

CTPGIWXHHEDIPGTSTEDHXITSIWGTGPXCHDUDCTLWDEDHHTHHTSQTPJI
NLXIWDJIKPCXINHIGTCVIWLXIWDJIXCHDATCRTRDJGPVTLXIWDJIUTGDH
XINPCSPAIIWTKXGIJTHDUBPCLXIWDJIWXHKXRTHIWXHEGPXHTLWXRW
LDJASQTJCBTPCXCVCUAPIITGNXUXCHRGXQTSKGTGWJBPCPHWTHXHJQJIP
YJHIIGXQJITIDIWTBTBDGNDUQDPIHLPXCPSDV

Inverse of alpha25 is : 25 and Beta is25

Code: Reference File -> KeyFinder2.java

Note, only important parts of the code are shown, a copy is attached to the assignment.

```

    try {
        orderingInput(displacement(list));
    } catch (IOException e) {
        e.printStackTrace();
    }
    int x = -1;
    while( ++x < 26){
        beta.add(x);
    }
    ArrayList<Integer> outa = new ArrayList<Integer>();
    ArrayList<Integer> outb = new ArrayList<Integer>();
    for( int x1 = 1;x1<alpha.length - 1;x1++){
        for( int j=1;j<beta.size();j++){
            int tmp = (alpha[x1]*letters.indexOf(freqs.first().letter)+beta.get(j))%26;

            if(tmp == letters.indexOf('E')){
                outa.add(alpha[x1]);
                outb.add(beta.get(j));
            }
        }
    }
    System.out.println("Possible Alpha List : " + outa);
    cout.write("Possible Alpha List : " + outa + "\n");
    System.out.println("Possible Beta List : " + outb);
    cout.write("Possible Beta List : " + outb + "\n");

    for(int x1 = 0; x1 < alpha.length; x1++){
        for( int y = 0; y < beta.size(); y++){
            int myAlph = alpha[x1];
            System.out.println("\nOutput: \n" + decrypt(inverse(myAlph), beta.get(y)));
            System.out.println("Inverse of alpha" + myAlph + " is : " + inverse(myAlph) + " and
Beta is" + beta.get(y));

            try{
                cout.write("\nOutput: \n" + decrypt(inverse(myAlph), beta.get(y))+"\n\n");
                cout.write("Inverse of alpha" + myAlph + " is : " + inverse(myAlph) + " and Beta
is" + beta.get(y)+"\n");
            } catch (Exception e){//Catch exception if any
                System.err.println("Error: " + e.getMessage());
            }
        }
    }

    try { cout.close(); } catch (IOException e) { e.printStackTrace();}

```

○ Substitution Cipher

ELNQYGEVQKYREMBEELVBHVJNMRBNDYBERMJZMQMDYRNWMGELMEWLNBMKNRGY
BDVNGLNBYBJFMKKNMGRGEYDVNLNVGGEVJJINRFQTCMLJVINVBELNKMGEYVEVG
INRFGVJJFZYRKNYKJNEYCRFMELVGZTBNRMJMJJQYQNBEGKMGEKRNGBEMBDZ
TETRNMJWMFGLMINNPVGENDMJWMFGWVJJNPVGEELNERNMJZMQMDYRVMBGCMBJY
YXMEMJJELNDVZ ZNRNBEQYQNBEGSTGEELMEWMFWNCMBJYXMEMGERNECLYZEL
NRYCXFQYTBEMVBGZYRVBGEMBCNELNFCMBGNNLYWKNRQMBNBEMJJELNQYQNB
GMRNMBDELNFCMBJYXMEMBFQYQNBELMEVBENRNNGEGELNQVEVGSTGEMBVJJT
GVYBWNLMINLNRNYBNMRELELMEYBNQYQNBZYZJYWGMBYELNRYBNJVXNANMDG
YBMGERVBHMBDELMEYBCNMQYQNBEGHYBNVEVGHYBNZYRNINRWLNBMMRMJZMQ
MDYRVMBGNNMGCYRKGNMJJLNELVBXGVGELMEELNDNMDKNRGYBVGVBAMDCYBDV
EVYBVBELNKMREVCTJMRQYQNBATEELMEELNGMQNKNRGYBVGSTGEZVBNVBKJN
BEFYZYELNRQYQNBEGBYWWLNBVQFGNJZLNMRELMEGYQNAYDFVGDNDMDVGQKJF
GLRTHMBDGMFWLMEELNERNMJZMQMDYRVMBGGMFMAYTEDNMDKNYKJNWLVLVGGY
VEHYNG

The steps we followed to decrypt are similar to the affine cipher, and these are as follows:

- 1) Same as Affine Cipher first step.
- 2) Same as Affine Cipher second step.
- 3) Same as Affine Cipher third step.
- 4) Same as Affine Cipher fourth step.
- 5) Create a pairSub method, which calculates not only the letter that is repeated the most in the text but also any possible combination of letters in the text such as 1,2,3 and others. This allowed us to be able to create a table in which collums and rows where titled by those unique letters that are in the ciphertext and then matched as shown in the output generated by this code. This way, we were able to guess missing letters by guessing what letter should the next if we replace the most common triple by THE, most common pair by HE, and most common letter by E.
- 6) Finally, a replace method was created. This takes in the CHARACTER you want to replace as well as the replacement character. Taking into account that each replacement letter will look lowercase in the output text, and each not replaced letter will be capitalized. This method returns a list that contains the text with the actual changes. This way, we were able to change and change characters till the entire text made sense. As a result, we came up with the right “smart guess” characters and obtained a very clear context of the ciphertext as it is shown below.

Example Output:

Note: a copy of the original output-file generated by this code is attached.

Alphabet Used:

[E, L, N, Q, Y, G, V, K, R, M, B, H, J, D, Z, W, F, I, T, C, P, X, S, A]

Highest Freq:

(Letter: N Count: 092)	(Letter: F Count: 018)
(Letter: E Count: 084)	(Letter: K Count: 018)
(Letter: M Count: 082)	(Letter: Z Count: 016)
(Letter: B Count: 062)	(Letter: W Count: 015)
(Letter: G Count: 060)	(Letter: C Count: 014)
(Letter: Y Count: 059)	(Letter: T Count: 013)
(Letter: V Count: 050)	(Letter: X Count: 06)
(Letter: L Count: 044)	(Letter: I Count: 06)
(Letter: R Count: 040)	(Letter: H Count: 06)
(Letter: J Count: 038)	(Letter: A Count: 05)
(Letter: Q Count: 030)	(Letter: S Count: 03)
(Letter: D Count: 023)	(Letter: P Count: 02)

Max Coincidences is:(Letter: N| Count: 092)

Ordered Frequencies:

(Letter: _EL Count: 010)	(Letter: _QY Count: 04)	(Letter: _GM Count: 02)
(Letter: _LN Count: 08)	(Letter: _MD Count: 03)	(Letter: _QM Count: 02)
(Letter: _JJ Count: 06)	(Letter: _YB Count: 03)	(Letter: _GN Count: 02)
(Letter: _GE Count: 06)	(Letter: _NM Count: 03)	(Letter: _CN Count: 02)
(Letter: _BE Count: 06)	(Letter: _NB Count: 03)	(Letter: _CL Count: 02)
(Letter: _VB Count: 06)	(Letter: _TG Count: 03)	(Letter: _CM Count: 02)
(Letter: _KN Count: 05)	(Letter: _BN Count: 03)	(Letter: _EG Count: 02)
(Letter: _LM Count: 05)	(Letter: _NR Count: 03)	(Letter: _TE Count: 02)
(Letter: _MJ Count: 05)	(Letter: _NG Count: 03)	(Letter: _KM Count: 02)
(Letter: _VG Count: 04)	(Letter: _MQ Count: 03)	(Letter: _YQ Count: 02)
(Letter: _MB Count: 04)	(Letter: _MR Count: 03)	(Letter: _ZY Count: 02)
(Letter: _QN Count: 04)	(Letter: _EE Count: 03)	(Letter: _FG Count: 02)
(Letter: _RN Count: 04)	(Letter: _EM Count: 03)	(Letter: _IN Count: 02)
(Letter: _KJ Count: 04)	(Letter: _RQ Count: 02)	(Letter: _MG Count: 02)
(Letter: _GY Count: 04)	(Letter: _WL Count: 02)	(Letter: _NV Count: 02)
(Letter: _EY Count: 04)	(Letter: _YR Count: 02)	(Letter: _DY Count: 02)
(Letter: _BD Count: 04)	(Letter: _VE Count: 02)	(Letter: _ND Count: 02)
(Letter: _ME Count: 04)	(Letter: _NA Count: 02)	(Letter: _YN Count: 01)
(Letter: _EV Count: 04)	(Letter: _GV Count: 02)	(Letter: _GG Count: 01)

Max Coincidences is:(Letter: _EL| Count: 010)

Ordered Frequencies:

(Letter: _YQN Count: 04)	(Letter: _ELM Count: 02)	(Letter: _EEL Count: 02)
(Letter: _LME Count: 03)	(Letter: _HMB Count: 02)	(Letter: _MBD Count: 02)
(Letter: _YXM Count: 03)	(Letter: _NMR Count: 02)	(Letter: _INR Count: 02)
(Letter: _MBJ Count: 03)	(Letter: _NRQ Count: 02)	(Letter: _NRG Count: 02)
(Letter: _QNB Count: 03)	(Letter: _GEM Count: 02)	(Letter: _MEW Count: 02)
(Letter: _ELN Count: 03)	(Letter: _NBE Count: 02)	(Letter: _MGE Count: 02)
(Letter: _WLN Count: 02)	(Letter: _LND Count: 02)	(Letter: _RMJ Count: 02)
(Letter: _GHY Count: 02)	(Letter: _MJJ Count: 02)	(Letter: _ELV Count: 02)

Max Coincidences is:(Letter: _YQN| Count: 04)

Continue . . .

Are You Done?

no

Please Enter the LETTER to replace

S

Please Enter the replacement letter

j

Descending Order of Repeated Letters:

Alphabet : [,N ,E ,M ,B ,G ,Y ,V ,L ,R ,J ,Q ,D ,F ,K ,Z ,W ,C ,T ,X ,I ,H ,A ,S ,P]

Alphabet : [,e ,t ,a ,n , , ,s , ,i ,l ,o ,d ,y ,r , ,f ,c ,w ,k ,u , ,b ,j ,h ,v , , ,p ,m , , ,g , , , , , , , , , , , ,]

themoStimportantthingilearnedontralfamadorewasthatwhenapersondiesheonly appearstodieheisstillverymuchaliveinthepastsoitisverysillyforpeopleto cr yathisfuneralallmomentspastpresentandfuturealwayshaveePistedalwayswille Pistthetralfamadorianscanlookatallthedifferentmom entsjustthatwaywecanlookatastretchoftherockymountainsforinstancetheycan seehowpermanentallthemomentsareandtheycanlookatanymomentthatintereststh emitisjustanillusionwehavehereonearththatonemomentfollowsanotheronelike beadsonastringandthatonceamomentisgoneitisgonefor everwhenatralfamadorianseesacorpseallhethinksisthatthedeadpersonisinbad conditionintheparticularmomentbutthatthesamepersonisjustfineinplentyofo thermomentsnowwhenimyselfhearthat somebodyisdeadis simplyshrugandsaywhatt h etralfamadorianssayaboutdeadpeoplewhichissoitgoes

Are You Done?

no

Please Enter the LETTER to replace

P

Please Enter the replacement letter

x

Descending Order of Repeated Letters:

Alphabet : [,N ,E ,M ,B ,G ,Y ,V ,L ,R ,J ,Q ,D ,F ,K ,Z ,W ,C ,T ,X ,I ,H ,A ,S ,P]

Alphabet : [,e ,t ,a ,n ,s ,o ,i ,h ,r ,l ,m ,d ,y ,p ,f ,w ,c ,u ,k ,v ,g ,b ,j ,x]

themoStimportantthingilearnedontralfamadorewasthatwhenapersondiesheonly appearstodieheisstillverymuchaliveinthepastsoitisverysillyforpeopleto cr yathisfuneralallmomentspastpresentandfuturealwayshaveexistedalwayswille xistthetralfamadorianscanlookatallthedifferentmom entsjustthatwaywecanlookatastretchoftherockymountainsforinstancetheycan seehowpermanentallthemomentsareandtheycanlookatanymomentthatintereststh emitisjustanillusionwehavehereonearththatonemomentfollowsanotheronelike beadsonastringandthatonceamomentisgoneitisgonefor everwhenatralfamadorianseesacorpseallhethinksisthatthedeadpersonisinbad conditionintheparticularmomentbutthatthesamepersonisjustfineinplentyofo thermomentsnowwhenimyselfhearthat somebodyisdeadis simplyshrugandsaywhatt h etralfamadorianssayaboutdeadpeoplewhichissoitgoes

Are You Done?

yes

Example Output: Reference file -> SUBS.java

Note: a copy of the original output-file generated by this code is attached.

```

    try {
        orderingInput(pairSub(new ArrayList<SUBS>(list),1));
    } catch (IOException e) {
        e.printStackTrace();
    }
    boolean stop = false;

    Scanner in = new Scanner(System.in);
    stop = false;
    while(!stop){
        System.out.println("Please Enter the LETTER to replace");
        cout.write("Please Enter the LETTER to replace\n");
        String tmp1 = in.next();
        cout.write(tmp1+"\n");
        System.out.println("Please Enter the replacement letter");
        cout.write("Please Enter the replacement letter\n");
        String tmp2 = in.next();
        cout.write(tmp2+"\n");

        replace(new ArrayList<SUBS>(tmp), tmp1, tmp2);
        System.out.println("Are You Done?");
        cout.write("Are You Done?\n");
        if( in.next().toLowerCase().charAt(0) == 'y')
            stop = true;
        else
            stop = false;
        cout.write((stop == true)? "yes\n": "no\n");
    }
    try {
        cout.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static ArrayList<SUBS> replace(ArrayList<SUBS> arrayList,
    String string, String string2) throws IOException {
    listr = new ArrayList<SUBS>(arrayList);

    for( SUBS s: arrayList){
        if( s.pair.equals(string))
            s.pair = string2;
        // System.out.println(s.pair);
    }
    // System.out.println(arrayList);

```

```

    int x = 0;
    if(temp.isEmpty()){
        while( x != listC.size()){
            temp.add(x, new SUBS(" "));
            x++;
        }
    }
    x = 0;
    System.out.print("Descending Order of Repeated Letters: \n Alphabet : [");
    cout.write("Descending Order of Repeated Letters: \n Alphabet : [");

    for( SUBS s: freqs){
        System.out.print(" " + s.pair);
        cout.write(" " + s.pair);

        if( s.pair.equals(string))
            temp.add(x, new SUBS(string2));
        x++;
    }
    System.out.println("]");
    cout.write("]\n");

    System.out.print(" Alphabet : [");
    cout.write(" Alphabet : [");
    for( SUBS s: temp){
        System.out.print(" " + s.pair);
        cout.write(" " + s.pair);
    }
    System.out.println("]\n");
    cout.write("]\n");

    int i = 0;
    while(i != arrayList.size()){

        if( i == arrayList.size()/3 || i == 2*(arrayList.size()/3)){
            System.out.println();
            cout.write("\n");
        }
        System.out.print(arrayList.get(i).pair);
        cout.write(arrayList.get(i).pair);
        i++;
    }
    System.out.println("\n\n");
    cout.write("\n\n\n");
    return listr;
}

```

3. The following cipher is a stream cipher version of the Vigenere cipher. (This cipher is from Stinson).

Given a keyword $k_1 k_2 \dots k_m$, turn this into the key sequence (or key *stream*) $z_1 z_2 z_3 \dots$ defined as follows

- For $1 \leq i \leq m$, $z_i = k_i$ and
- For all $i \geq 1$, $z_{i+m} = (z_i + 1) \bmod 26$;

For example, the keyword ayc generates the key sequence aycbzdcaedbfecgfdhgeihfj Given a plaintext string $x_1 x_2 x_3 \dots x_n$, encrypt this by replacing each plaintext symbol x_i by ciphertext symbol $(x_i + z_i) \bmod 26$.

In this question, you may assume that the plaintext is in English.

3. Can you modify the Kasiski test to determine the length of the keyword? Briefly explain your answer.

Yes, the length of the example cipher text can easily be found by counting the distance of those letters that are repeated the most and finding its common divisor. This can easily be done with help of my code as shown below.

Ordered Frequencies:

(Letter: _f Count: 03)	(Letter: _b Count: 02)
(Letter: _e Count: 03)	(Letter: _a Count: 02)
(Letter: _d Count: 03)	(Letter: _j Count: 01)
(Letter: _c Count: 03)	(Letter: _i Count: 01)
(Letter: _h Count: 02)	(Letter: _z Count: 01)
(Letter: _g Count: 02)	(Letter: _y Count: 01)

Now, we proceed to split the cipher text into triple letters.

ayc bzd cae dbf ecg fdh gei hfj

For f:

f, 11 -> 3 -> 1 3
 f, 15 -> 6 -> 1 2 3 6
 f, 12 -> 12 -> 1 2 3 4

For e:

f, 8 -> 3 -> 1 3
 f, 12 -> 6 -> 1 2 3 6
 f, 19 -> 12 -> 1 2 3 4

For d:

f, 2 -> 3 -> 1 3
 f, 6 -> 6 -> 1 2 3 6
 f, 13 -> 12 -> 1 2 3 4

And so on

As we can see, they all share the same common factor of 3.

Therefore, the length of the key is three, which is proven above = ayc.

4. Describe how you can use the concept of index of coincidence to first determine the length of the keyword, and then to actually find the keyword.

We can use the Index of Coincidence to obtain the probability that two letters chosen at random from the cipher text are the same letter. To do this, we need to do the following steps.

First, measure of roughness. We need to estimate the MR from the given Cipher Text. This can be done in the following manner:

Second, find the total amount of frequencies N in the given cipher text by $N = \sum_{i=A}^{i=Z} Fi$

Third, find the number of letter containing the letter i th by $Pi = \frac{Fi(Fi-1)}{n(n-1)}$, and the formula is: $MR + 0.038 = \sum_{i=A}^{i=Z} pi^2$. Fi is the number of frequency that a letter has in a given cipher text.

To obtain the MR we just need to calculate $MR = \sum_{i=A}^{i=Z} pi^2 - 0.038$.

Finally, use the estimate MR formula to get the index of Coincidence $IC = MR + 0.038$. Once calculated, this result is added to the MR to obtain the IC value.

5. Test your method from part (2) on the following ciphertext. Give the plaintext, the key, and a clearly written description of the steps you followed to decrypt the ciphertext. This should include the actual values of indices of coincidence that you computed. If you wrote code to help you decrypt, you should also include a hardcopy of your source code.

WBGZVJEWJISRJFXGPLMIQROXWGXHSNNEFOLWDVQKBQOIA
UBNXZSJKTQBIAQLVXWQXCGLPTGHOTYNWGOHOWJXMIEJHH
NXIDDMBUTWTLJBHNHQRORMJKRQUCYULKUGBHTEXRBQXUJ
EOWCWRXWBAEDCHMZIOQEJOPPVVWYYNUSAOANXFLUMKUP
ZJDVACISDPKHHULZAECTPVEZPZFXPYVOVNWZXQGTGKJKA
CTHBGHIRTEEEEMSPUEHQUEHSGMHTQDHIIOIAWBGZEZDN
DZMVWNAEAGPDNJAEOKBREKLXQDNVRNEJBPFCCMKMTNOZU
OIHVETNICAFWRAFGHMULGHPWICJKCPOOSZVMXRBBFCWRY
UUFPLHLCCJGGWFETZJOVWXNNAYNATVQOJHQDZJKUZXA
KDKLSRTHKJNGLKMLBDFXOHUYNZVZGZWWMLHJRQJRNNKTE
KWQLRAGIQHVZAYGKRWDKDKV

To decrypt the above cipher text we took the following approach:

First, use a similar function as that used in the other problems to find the most frequent letters.

Second, split the cipher text in different groups to calculate the index of coincidence. After divided in groups each column was tested in the MR method to get the different ICS according to the m values (divisions). Another way was by shifting a copy of the cipher text m times, the shifting that produced more combinations of repeated words is the index selected.

Third, each column was tested to get the letter that was repeated the most which subtracted the letter that is repeated the most in the English alphabet should give a key letter.

Finally, each key letter is appended to form a key word and used to decrypt the cipher text. We created a cipher text method, which was already tested with the example provided. Unfortunately, something did not work out well in our approach and we are not obtaining the desired output. However, we are convinced that our approach is close enough to lead to the solution.

Example Output:

Note: a copy of the original output-file generated by this code is attached.

[N, H, D, Z, P, V, Y, H, E, E, M, I, F, O, B, U, C, G, N, Q, W, N]
Letters Repeated: [N, H, D, Z, P, V, Y, E, M, I, F, O, B, U, C, G, Q, W]
Time No Repeated: [3, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Number of Coincidences: 22

Continue . . .

[G, K, Q, I, A, X, X, B, K, C, H, E, M, H, D, D, N, E, C, X, C, T, K, K, H, R]
Letters Repeated: [G, K, Q, I, A, X, B, C, H, E, M, D, N, T, R]
Time No Repeated: [1, 4, 1, 1, 1, 3, 1, 3, 3, 2, 1, 2, 1, 1, 1]
Number of Coincidences: 26

Continue . . .

[W, I, A, E, E, V, N, F, W, J, K]
Letters Repeated: [W, I, A, E, V, N, F, J, K]
Time No Repeated: [2, 1, 1, 2, 1, 1, 1, 1, 1]
Number of Coincidences: 11

Displacement: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]
Coincidences: [22, 22, 19, 21, 14, 16, 13, 17, 16, 18, 13, 14, 26, 14, 16, 19, 17, 16, 18, 17, 20, 13, 14, 11]
Key length is:13
Max Coincidences is:26

Code: Reference Code -> KeyFinder.java & IOC.java

Note, only important parts of the code are shown, a copy is attached to the assignment.

```

    int x = 1;
    ArrayList<Integer> tmp = new ArrayList<Integer>();
    while(x != 25){
        tmp.add(x);
        System.out.println("\n");
        System.out.println("Letters Repeated: " + displacement(x));
        System.out.println("Time No Repeated: "+cnt);
        count.add(x-1, coincidences());
        System.out.println("Number of Coincidences: " + count.get(x-1));
        x++;
    }

    System.out.println("\n\nDisplacement:" + tmp);
    System.out.println("Coincidences:" + count);

    int loc = getMaxI(count)+1;
    System.out.println("Key length is:" + loc );
    System.out.println("Max Coincidences is:" + count.get(loc-1));
    // System.out.println(split(list.toString(), loc));

    LinkedList<KeyFinder> tp = new LinkedList<KeyFinder>();
    list.getFirst().cds = -1;
    KeyFinder first = list.getFirst();

    do{
        try {
            tp.addAll(orderingInput((pairSub(new
ArrayList<KeyFinder>(list), 3)))));
        } catch (IOException e) {
            e.printStackTrace();
        }
        // System.out.println("Locations: " + location);
        list.addFirst(list.removeLast());
    } while(!list.getFirst().equals(first));

```